

Union-Find 与集合划分

动态合并后，怎样快速回答“是否同一集合”

408 · 数据结构 Topic DS-06

母问题：动态连通不是一般树，而是代表元问题

并查集的生成链是

Partition → Representative → Find/Union → Path Compression + Rank → Dynamic Connectivity.

每个集合用一棵父指针树表示，但树形只是实现手段；真正的抽象是分区。find(x) 返回集合代表元，union(a,b) 合并两个代表元不同的集合。任何操作后，父指针必须最终通向唯一根，根的数量必须等于集合数量。

本册定位与 Owner 边界

- **Owns**：MakeSet、Find、Union、代表元、按秩/大小合并、路径压缩与动态连通语义。
- **Uses**：数组下标和成本向量；向 DS-B03/DS08 输出 ‘connected/unite’ 接口。
- **Stops**：一般树遍历、Kruskal 的边权排序与生成树正确性由其他 Owner 负责。

目录

1	分区不变量	2
2	为什么需要两种局部修复	2
2.1	朴素合并的最坏形状与两种优化的分工	2
2.2	代表元不是业务关键字	2
2.3	动态连通与 Kruskal 的最小接口	3
3	代码：代表元、路径压缩与按秩合并	3
3.1	测试：重复合并、压缩与空全集	4
4	复杂度与边界	6
5	做题控制协议与压缩复原	6

1 分区不变量

初始时每个元素自成集合, $parent[i] = i$ 。父指针反复追踪必到根; 根是自己的父节点。若两个元素根相同, 则属于同一集合; 若根不同, 合并时把一个根挂到另一个根下, 并将集合数减一。重复合并不改变分区。

并查集的“集合”是一个随操作序列变化的等价关系: 自反、对称、传递性始终成立。find 只返回代表元, 不改变等价类; 路径压缩虽会改写多个 parent 字段, 但改写前后的根相同, 因此不改变答案。union 必须先 Find 两端再合并根, 不能把任意节点直接挂到任意节点, 否则可能破坏“只有根才代表集合”的不变量。

若采用一个数组同时保存父指针和大小, 可令根节点存负的集合大小、非根节点存父下标: $parent[root] = -size$, 合并时把较小集合的根挂到较大集合, 并更新负值。这省去单独的 rank/size 数组, 但比较时必须只对根读取负值, 非根位置的数值没有大小意义。初始化、越界和 $n = 0$ 都应作为独立边界测试。

2 为什么需要两种局部修复

若只把一个根挂到另一个根下, 连续合并可能生成链, 使 Find 退化到 $O(n)$ 。按秩合并总把较矮树挂到较高树, 限制树高; 路径压缩在 Find 返回时让沿途节点直接指向根, 摊还复杂度达到近似常数 $O(\alpha(n))$ 。

2.1 朴素合并的最坏形状与两种优化的分工

如果每次都令 $root_a.parent = root_b$, 合并序列可以把元素串成一条链:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow (n-1).$$

此时一次 Find 可能追踪 n 条父边。按秩合并保存的是“树高的上界”或等价的规模信息: 只有当两棵树秩相同时, 合并后新根的秩才增加一; 把低秩根挂到高秩根不会破坏上界。按大小合并使用集合元素数代替秩, 证明思路相同。

路径压缩保存的是“这一次访问已经发现了代表元”。递归版在回溯时把路径上每个节点直接指向根; 迭代版可以先记录路径再统一改父指针。它不改变集合划分, 只改变实现树形。两种优化解决不同问题: 秩/大小合并限制新树长高, 路径压缩摊薄反复访问的旧路径; 只使用其中一种仍正确, 但复杂度证明不同。

按秩合并的高度证明是对数级: 只有两棵相同秩的树合并时秩才加一, 而秩为 r 的根至少包含 2^r 个节点, 因此 $r \leq \lfloor \log_2 n \rfloor$ 。仅有按秩 (不压缩) 时单次 Find 为 $O(\log n)$; 仅有路径压缩 (不按秩) 时可在特定序列下变差; 两者同时使用时, 任意 m 次操作总时间为 $O(m\alpha(n))$, 其中反阿克曼函数在实际输入范围内极小, 但它不是严格的常数承诺。

2.2 代表元不是业务关键字

代表元只是实现中回答“是否同集合”的规范身份, 合并后可以发生变化。题目若要求某个特定元素继续作为代表元, 必须额外维护代表策略; 普通按秩合并不保证“较小编号为根”。并查集也不保存集合内部顺序、路径长度或成员列表; 如果需要枚举成员、撤销合并或维护权值, 应另建结构, 不能从 parent 数组反推不存在的业务信息。

2.3 动态连通与 Kruskal 的最小接口

把无向边按某种顺序处理时，若边两端 Find 结果不同，加入该边不会在当前森林中形成环，并执行 Union；若代表元相同，则该边会连接同一连通分量内部，应拒绝。Kruskal 还需要按边权排序、停止于 $n - 1$ 条边并证明得到最小生成树，这些由 DS08 Own；DS06 只保证每次“同组判定—合并”状态转移正确。

若题目只问连通分量数量，初始为 n 个集合，每次成功合并减一，重复边不减。若题目给自环或重复边，Find 会返回同一代表元，正好把这些边识别为不改变分区的事件。不要把“边数达到 $n - 1$ ”当成无条件连通；只有成功合并次数达到 $n - 1$ 且不存在被拒绝的分支时，才能推出一棵生成树。

并查集适合“只增不删”的无向连通性：边一旦合并就不能直接撤销，不能回答带时间顺序的删除边问题，也不能返回两点之间的具体路径、距离或最短路。若问题要求这些信息，应改用 BFS/DFS、动态树、离线回滚或其他结构，并明确它们的 Owner。并查集也不区分有向边方向；把有向图的可达性直接交给 Union-Find 会把方向信息丢失。

3 代码：代表元、路径压缩与按秩合并

完整实现位于 code/ds06_union_find.hpp，测试位于 code/ds06_union_find_test.cpp；正文直接引用真实源码。

```
1  public:
2      explicit UnionFind(std::size_t count)
3          : parent_(count), rank_(count, 0), components_(count) {
4          for (std::size_t i = 0; i < count; ++i) {
5              parent_[i] = i;
6          }
7      }
8
9      std::size_t size() const { return parent_.size(); }
10     std::size_t components() const { return components_; }
11
12     std::size_t find(std::size_t value) {
13         check(value);
14         if (parent_[value] != value) {
15             parent_[value] = find(parent_[value]);
16         }
17         return parent_[value];
18     }
19
20     bool connected(std::size_t left, std::size_t right) {
21         return find(left) == find(right);
22     }
23
24     bool unite(std::size_t left, std::size_t right) {
25         std::size_t root_left = find(left);
26         std::size_t root_right = find(right);
27         if (root_left == root_right) {
28             return false;
29         }
30         if (rank_[root_left] < rank_[root_right]) {
```

```

31     std::swap(root_left, root_right);
32 }
33 parent_[root_right] = root_left;
34 if (rank_[root_left] == rank_[root_right]) {
35     ++rank_[root_left];
36 }
37 --components_;
38 return true;
39 }
40
41 bool invariant() const {
42     if (components_ > parent_.size() || parent_.size() != rank_.size()) {
43         return false;
44     }
45     std::size_t roots = 0;
46     for (std::size_t i = 0; i < parent_.size(); ++i) {
47         if (parent_[i] >= parent_.size() || rank_[i] > parent_.size()) {
48             return false;

```

代码清单 1: Find 的路径压缩与 Union 的按秩合并

‘find’ 的递归返回值就是代表元；回溯赋值是路径压缩。‘unite’ 先分别 Find，再比较秩，最后只减少一次 ‘components’。根相同的分支必须提前返回，否则重复合并会错误减少集合数。

```

1     if (parent_[i] == i) {
2         ++roots;
3     }
4 }
5 return roots == components_;
6 }
7
8 private:
9 void check(std::size_t value) const {
10     if (value >= parent_.size()) {
11         throw std::out_of_range("union-find element out of range");
12     }
13 }
14
15 std::vector<std::size_t> parent_;
16 std::vector<std::size_t> rank_;
17 std::size_t components_ = 0;
18 };
19
20 } // namespace ipara::ds06
21
22 #endif

```

代码清单 2: 分区不变量和越界边界

3.1 测试：重复合并、压缩与空全集

```

2 void test_make_set_and_bounds() {
3     UnionFind sets(4);
4     assert(sets.size() == 4 && sets.components() == 4 && sets.invariant());
5     bool threw = false;
6     try {
7         sets.find(4);
8     } catch (const std::out_of_range&) {
9         threw = true;
10    }
11    assert(threw);
12 }
13
14 void test_union_and_duplicate_union() {
15     UnionFind sets(6);
16     assert(sets.unite(0, 1));
17     assert(sets.unite(2, 3));
18     assert(sets.unite(1, 2));
19     assert(!sets.unite(0, 3));
20     assert(sets.components() == 3);
21     assert(sets.connected(0, 3));
22     assert(!sets.connected(0, 4));
23     assert(sets.invariant());
24 }
25
26 void test_path_compression_reaches_representative() {
27     UnionFind sets(8);
28     sets.unite(0, 1);
29     sets.unite(2, 3);
30     sets.unite(4, 5);
31     sets.unite(6, 7);
32     sets.unite(0, 2);
33     sets.unite(4, 6);
34     sets.unite(0, 4);
35     const std::size_t representative = sets.find(7);
36     assert(representative == sets.find(0));
37     assert(sets.invariant());
38 }
39
40 void test_empty_universe() {
41     UnionFind sets(0);
42     assert(sets.components() == 0 && sets.invariant());
43 }
44
45 int main() {
46     test_make_set_and_bounds();

```

代码清单 3: 代表元一致性和集合数变化测试

```

1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
2   code/ds06_union_find_test.cpp -o /tmp/ds06_test
3 /tmp/ds06_test

```

4 复杂度与边界

操作	摊还复杂度	保证	边界
Find/Union	$O(\alpha(n))$	路径压缩 + 按秩/大小	空集合、越界
MakeSet	$O(n)$	每项自为根	$n = 0$ 合法
空间	$O(n)$	parent/rank 两数组	不保存集合内部顺序

不要把“父指针树”当作业务树

并查集不支持按层遍历、输出路径或保持关键字有序；它只回答代表元和同集合关系。把 Union-Find 的父树画成一般树来做遍历，会把实现结构误当成目标对象。

5 做题控制协议与压缩复原

先写当前分区和每个根；每次合并只比较两个代表元；重复合并检查是否同根；若问复杂度，指出路径压缩和秩策略共同作用。忘掉代码后复原：**集合如何表示？谁是代表元？Find 是否压缩？Union 挂哪棵树？集合数何时减少？**

跨册交接

DS06 向 DS-B03 和 DS08 只提供 ‘connected/unite’。Kruskal 的排序、选边和无环性证明由 DS08 Own；本册只保证每次加入边前后动态分区不变量。

旧总册关于代表元、路径压缩、按秩合并和 Kruskal 接口的内容已吸收；带权并查集、可回滚并查集和持久化版本作为 Extension。